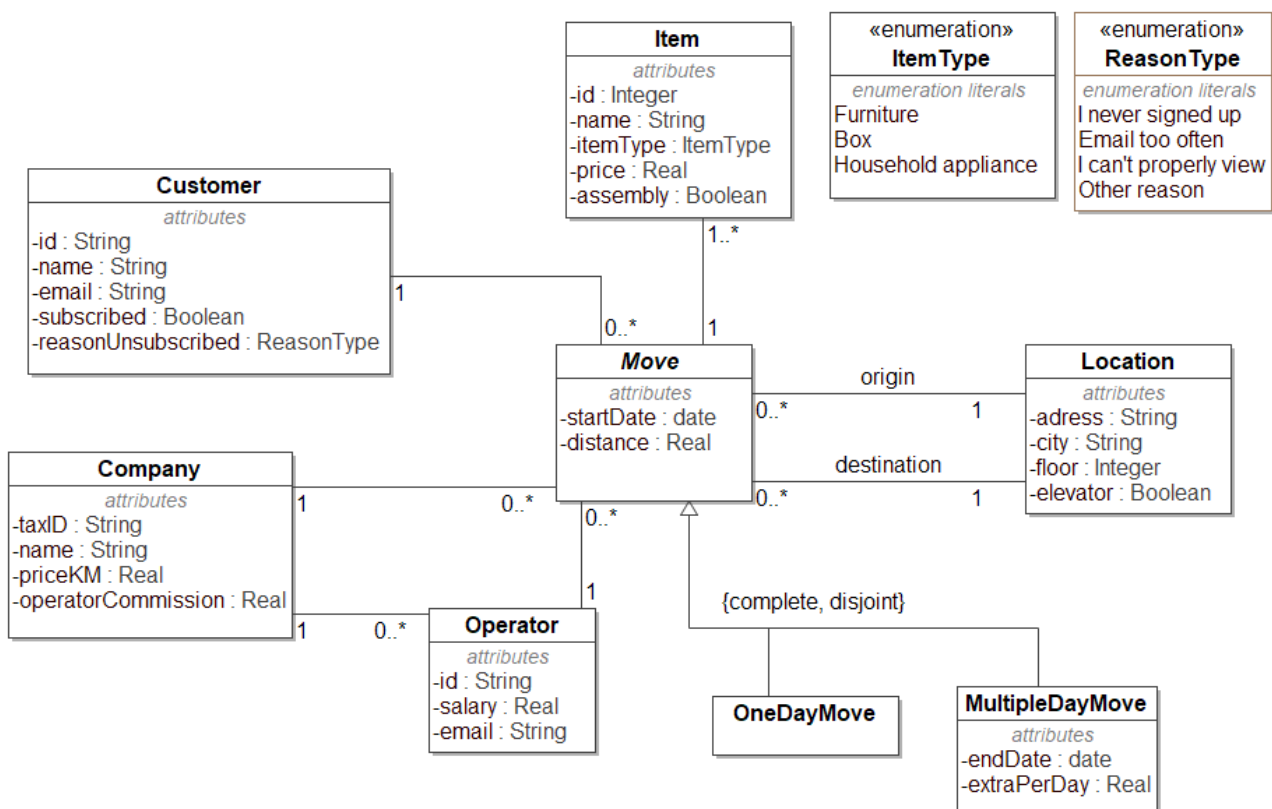


Análisis y Diseño con Patrones

Práctica 2: Patrones de Diseño y de Asignación de Responsabilidades

La asociación que nos contrató en la Práctica 1 para desarrollar un software para poner en contacto clientes con empresas de mudanzas quiere que añadamos nuevas funcionalidades al software. A continuación disponéis del diagrama estático de análisis de la Práctica 1:

Diagrama estático de análisis



Claves de las clases:

- *Customer*: id
- *Operator*: id
- *Location*: address

- *Company: taxId*
- *Item: Id*
- *Move: id(Customer) + originAdress + startDate*

Restricciones de integridad:

- La fecha de inicio de una mudanza debe ser anterior a la fecha final de MultipleDayMove.
- La dirección de origen no puede ser la misma que dirección destino.
- Un operario no puede tener mudanzas solapadas.
- Todos los enteros y reales son números positivos.
- La empresa que hace la mudanza es la misma en la que trabaja el operario que participa.

Nuevos atributos del diagrama en esta Práctica:

- City: String en la clase Location.
- Email: String en la clase Operator.

Ejercicio 1 (15%)

Supongamos que la asociación que se encarga de poner en contacto las diferentes empresas con los clientes quiere registrar en el sistema el % de beneficio que cobrará a cada empresa por las mudanzas conseguidas a través de su sistema. Este % es único, pero queremos poder cambiarlo. Se pide:

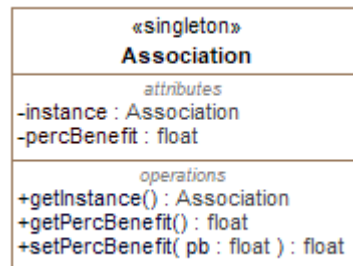
- ¿Qué patrón de diseño es adecuado para registrar este % de beneficio?
- Proporciona el diagrama de clases resultante de aplicar este patrón (sólo la parte de la aplicación del patrón).
- Escribe el pseudocódigo del diagrama de clases que propones en el apartado anterior.

Nota:

- No hace falta que tengáis en cuenta el cálculo del % de beneficio para este ejercicio.

Solución:

- a) Aplicaremos el patrón Singleton a la clase Asociación para mantener una única instancia de esta clase que tenga definido el % de beneficio que quiere obtener sobre cada empresa. Este % de beneficio se guardará una única vez para todas las empresas.
- b) El diagrama de clases resultante del patrón tendrá definida una única clase.



c)

```
public class Association {

    private static Association instance = null;
    private float percBenefit;
    private Association() {}

    public static Association getInstance() {
        if (instance == null) {
            instance = new Association();
        }
        return Association;
    }

    public float getPercBenefit() {
        return percBenefit;
    }

    public float setPercBenefit(pb) {
        percBenefit = pb;
    }

}
```

Ejercicio 2 (15%)

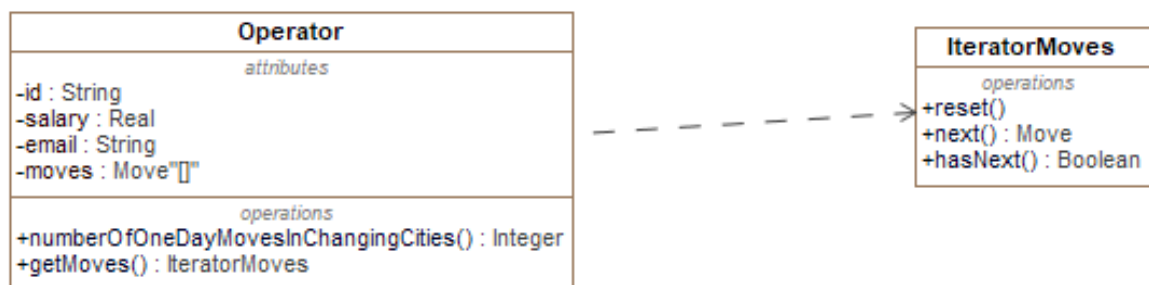
Supongamos ahora que queremos definir una operación

numberOfOneDayMovesChangingCities():Integer en la clase *Operator* que devuelve el número de mudanzas de un día en las que ha participado un operador y tienen diferente ciudad de origen que de destino. En este momento del diseño del sistema todavía no sabemos qué estructura de datos utilizaremos para guardar las mudanzas en las que ha participado un operador. Se pide:

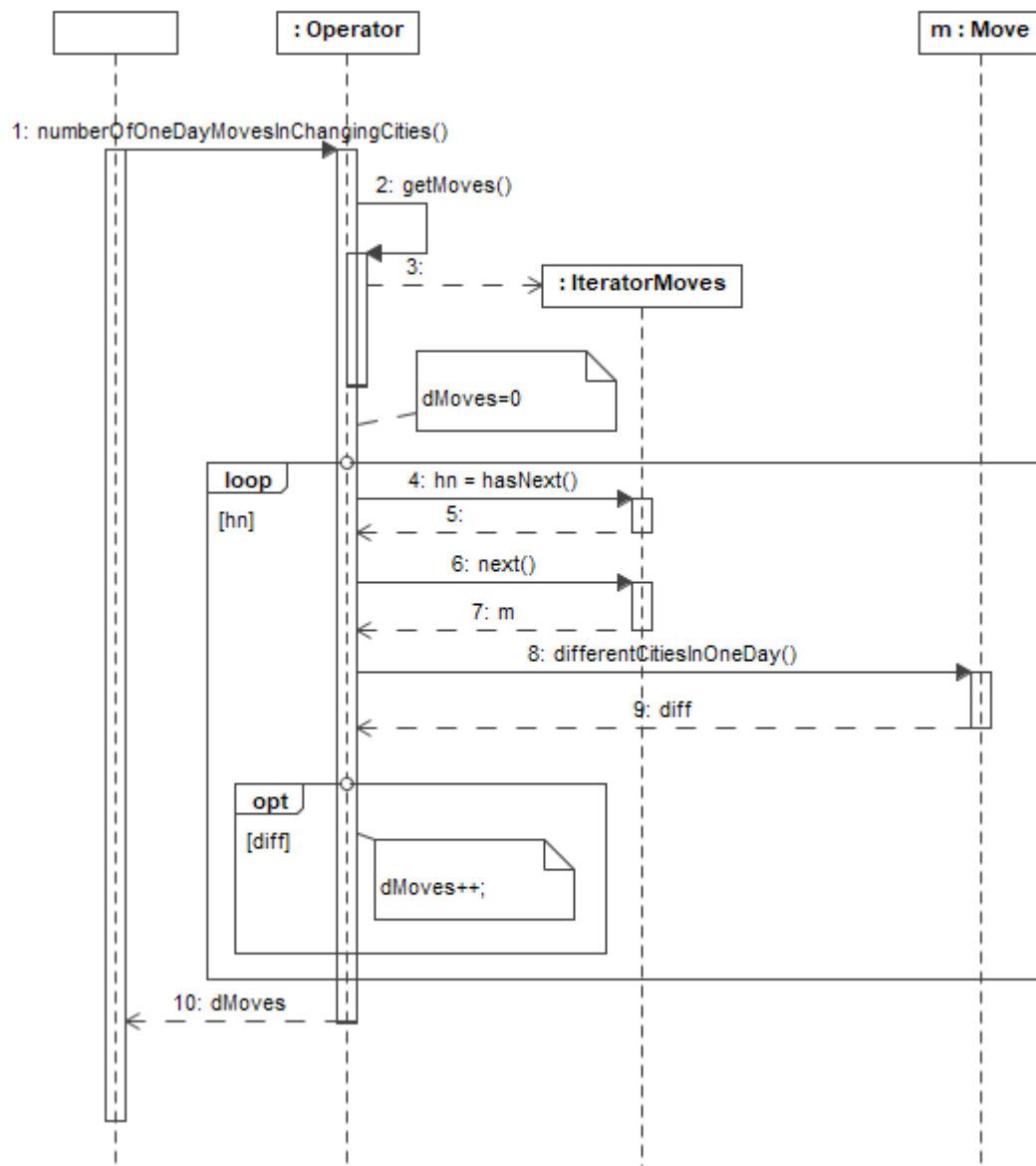
- ¿Qué patrón de diseño utilizarías para diseñar esta operación? Justifica tu respuesta.
- Muestra la parte del diagrama de clases que se ve modificada como resultado de la aplicación del patrón utilizado.
- Muestra el diagrama de secuencia o el pseudocódigo de la operación *numberOfOneDayMovesInChangingCities():Integer* de la clase *Operator*.

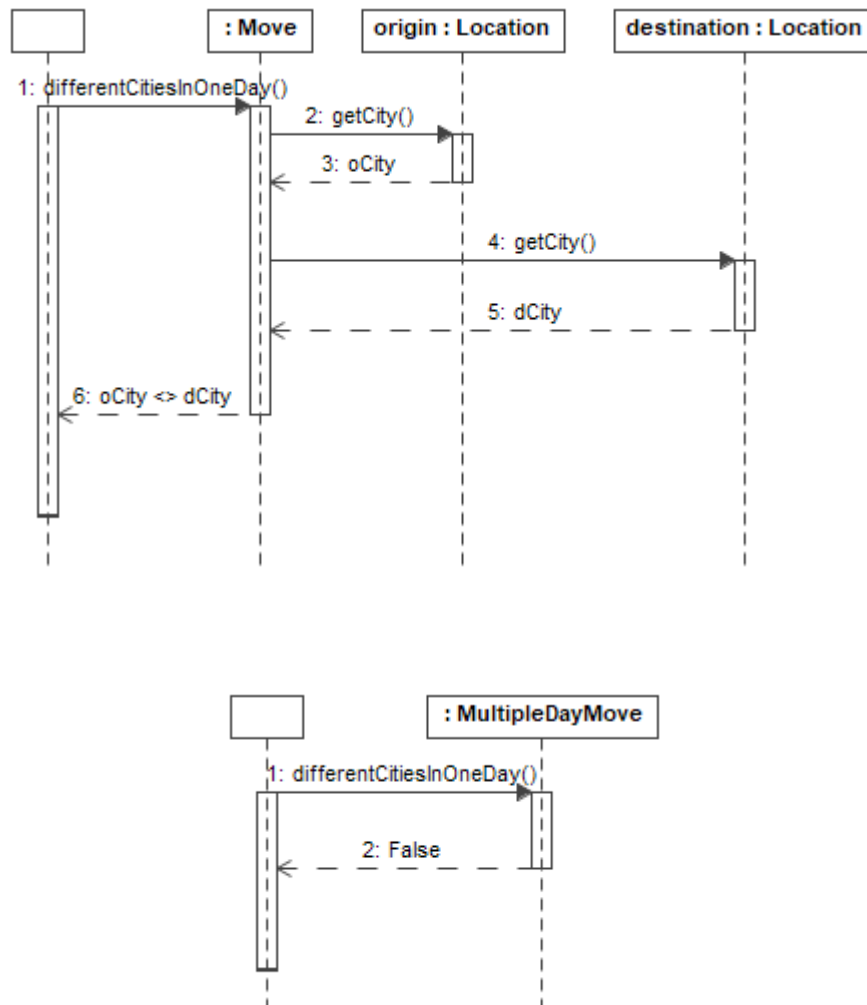
Solución:

- Aplicaremos el patrón Iterador que permite recorrer las mudanzas en las que ha participado un operador encapsulando la estructura interna de esta información.
- El diagrama muestra la clase *Operator* y el iterador de las mudanzas para recorrer las mudanzas en las que ha participado un operador.



- El diagrama de secuencia muestra el recorrido de las mudanzas en las que participa un operador mediante el iterador. Para cada mudanza se comprueba que la ciudad de origen es diferente que la de destino y si es así y es mudanza de un solo día se suma uno al contador.





Ejercicio 3 (20%)

Cuando los clientes quieren añadir objetos (items) a su mudanza se tienen que crear instancias de estos, por lo tanto, ahora vamos a diseñar esta operación llamada *createItemInMove(name: String, itemType: itemType, assembly: Boolean):Integer* de la clase *Move*. Esta operación, que crea un objeto para añadir a la mudanza, debe hacer lo siguiente:

- Crear un objeto (*Item*) y asignarle un nombre, tipo de objeto y si necesita montaje. El id y el precio lo asignará automáticamente el sistema (podéis obviar estos dos campos).
- Guardar el objeto creado en la mudanza (asumimos que las mudanzas registran el conjunto de items pero que los items no registran la mudanza a la que pertenecen).

- Calcular y devolver el número total de objetos que sean del mismo tipo (*itemType*) que el que creamos y tenga hasta el momento la mudanza.

Ejemplo: tenemos una mudanza con 2 muebles y 1 caja, entonces llamamos a la función *createItemInMove(desk,furniture,true)*, para añadir el escritorio (i.e., desk) que es un mueble (i.e., furniture) y nos devuelve el número 3 (que se corresponde con los dos muebles que ya teníamos más el mueble que acabamos de crear en nuestra mudanza).

Se pide:

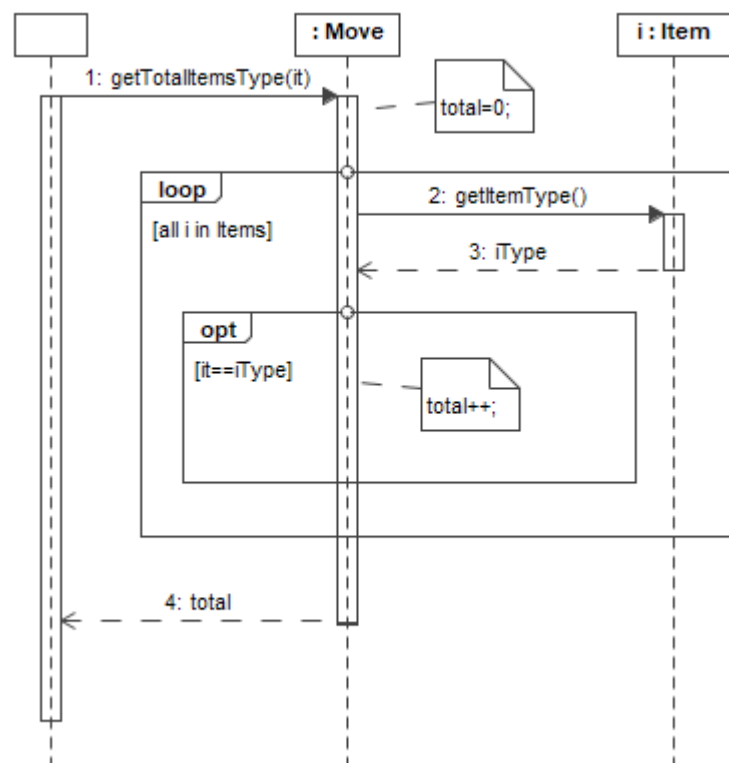
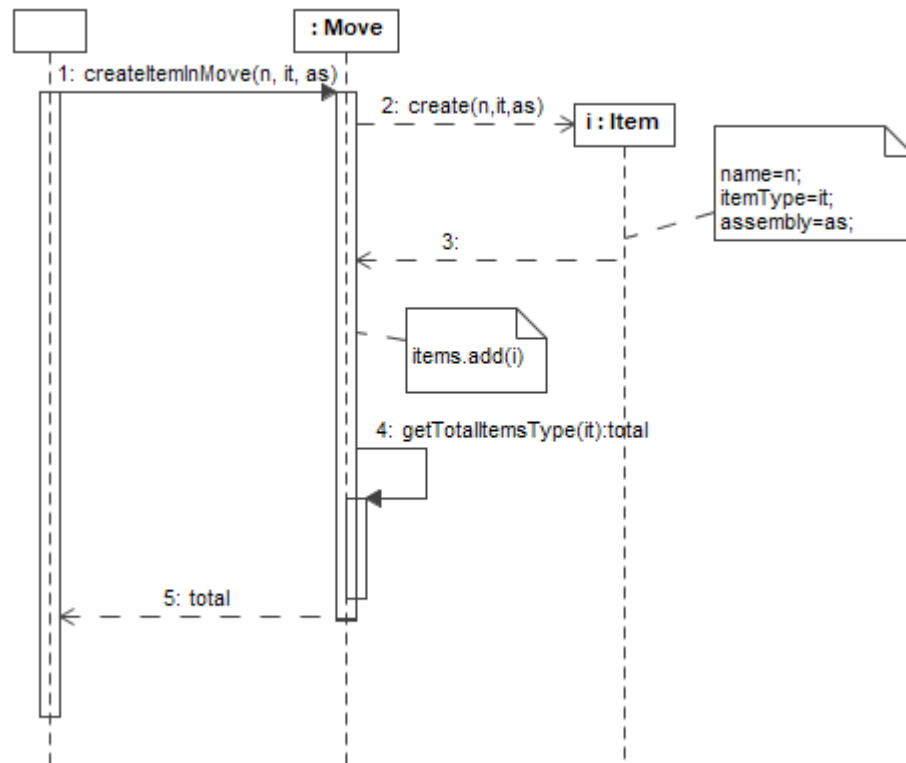
- ¿A qué clase corresponde la responsabilidad de implementar el cálculo del número total de objetos con el mismo tipo de objeto que tiene el cliente registrados en una mudanza? Justifica la respuesta.
- ¿Qué patrones de diseño son adecuados para aplicar en esta situación?
- Haz el diagrama de secuencia o el pseudocódigo de la operación *createItemInMove* y de todas las operaciones que sean invocadas desde esta operación.

Nota:

- Nos dicen que el cálculo del número total de objetos con el mismo tipo de objetos que tiene una mudanza también es necesario para otras operaciones del sistema que estamos desarrollando. Se valorará especialmente que vuestra solución sea reusable.

Solución:

- La clase que contiene la información necesaria para poder realizar este cálculo es la clase *Move*. Esta clase tiene definidos los objetos que el cliente ha registrado en su mudanza y, por lo tanto, aplicando el patrón Experto es la clase donde se tiene que definir este cálculo.
- Tenemos que aplicar el patrón Creador y asignar a la clase *Move* la responsabilidad de crear un objeto ya que esta clase contiene los objetos.
- A continuación se muestra el diagrama de secuencia de la operación *createItemInMove*. Notad que se ha creado una operación para realizar el cálculo del número total de objetos del mismo tipo de objeto que tiene una mudanza para que esta operación pueda ser reusada en otras operaciones del sistema.



Ejercicio 4 (25%)

Cada compañía quiere saber el número de KMs acumulado en sus mudanzas. Este acumulado se calcula con la operación *getAccumulatedKM():Real* de la clase *Company*. Para calcular estos KM la compañía hace la suma de todos los KM por mudanza realizada. Los KM realizados por mudanza se calculan de la siguiente manera:

- Si es una mudanza de un día los KM serán la distancia que marca la mudanza por dos (ida y vuelta) por el número de viajes necesarios. Este número de viajes puede ser 1 o 2. Si en la mudanza se tienen que mover más de 50 objetos y la distancia es menor que 100KM entonces se harán 2 viajes. Si no se cumplen los dos requisitos anteriores se hará en 1 solo viaje.
- Cuando se trate de una mudanza de varios días los KM serán la distancia que marca la mudanza por dos (ida y vuelta) por el número de días que dura la mudanza (*endDate* - *startDate* + 1). (En este tipo de mudanza no hay opción de hacer varios viajes.)

Se pide:

- a) ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- b) Haz el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar solo la parte de aplicación del patrón).
- c) Haz el diagrama de secuencia o el pseudocódigo de la operación *getAccumulatedKM():Real* de la clase *Company* y de todas las operaciones que sean invocadas desde la operación.

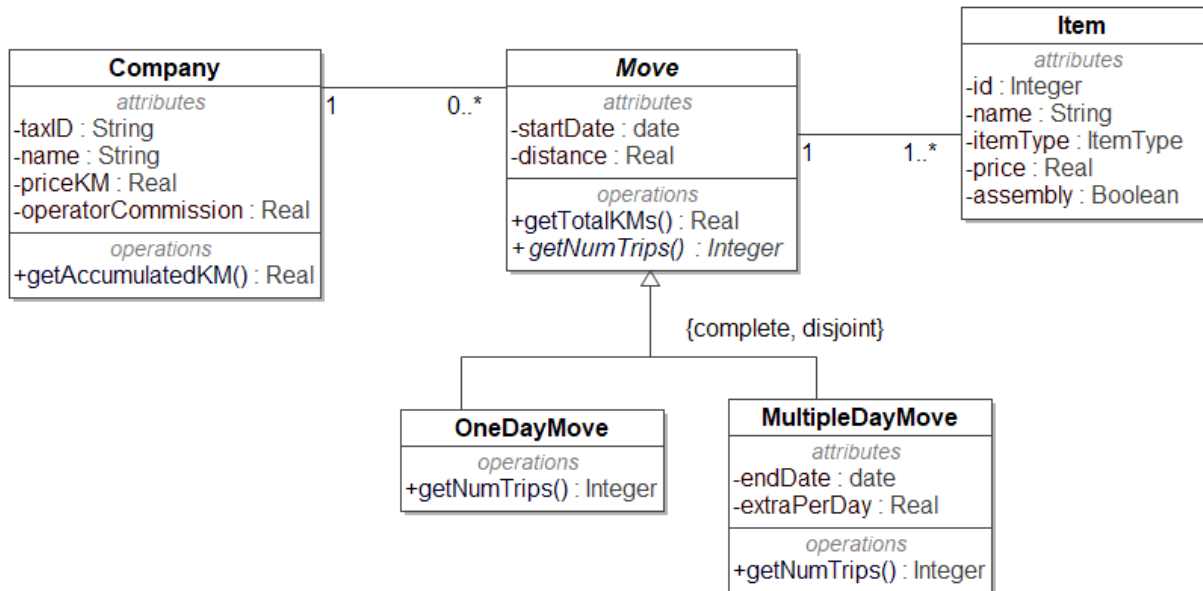
Nota:

- Se valorará especialmente que en la solución propuesta no haya repetición de código.

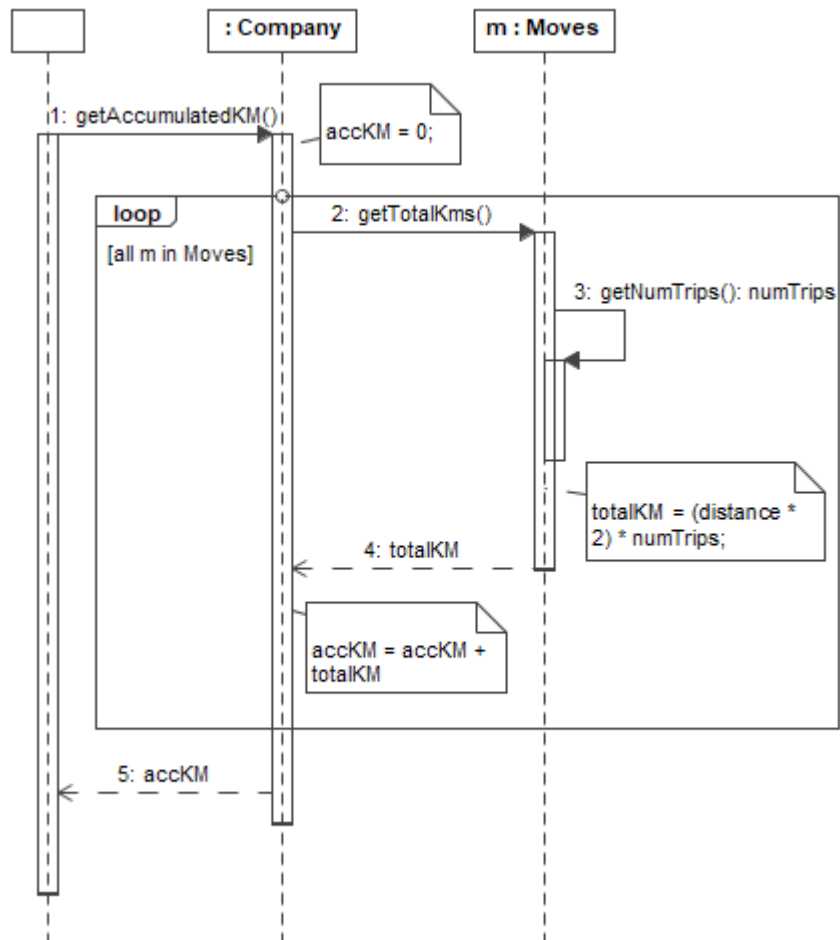
Solución:

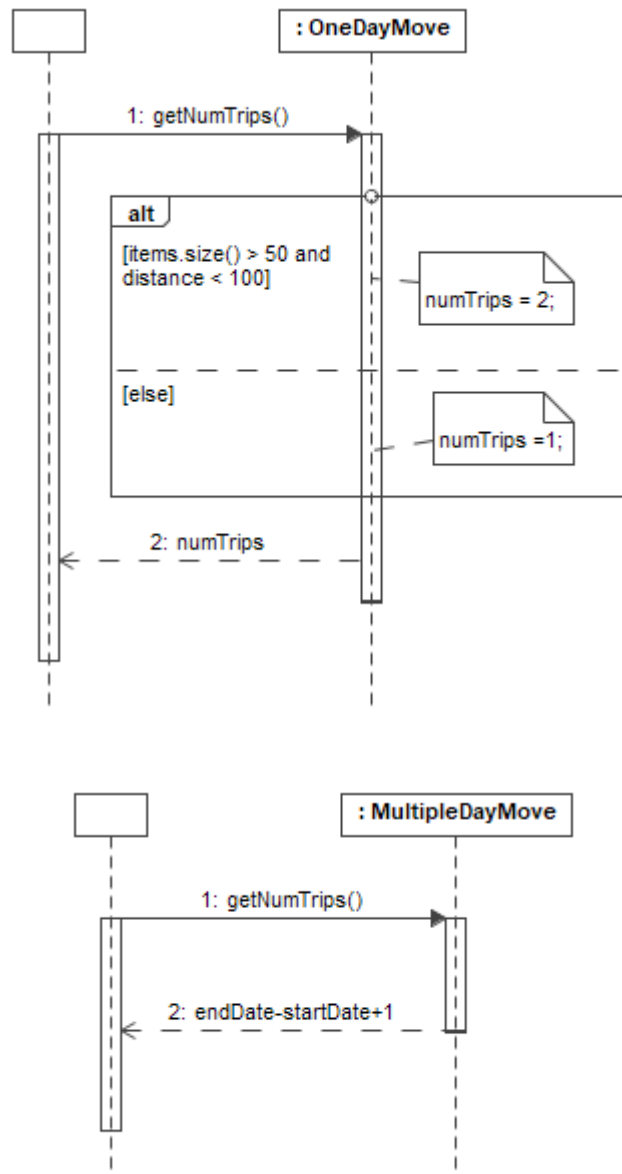
- a) Aplicaremos el patrón método Plantilla para diseñar la operación *getAccumulatedKM()* que calculará los KMs de todas las mudanzas de una compañía. La parte común corresponde a los $KMs \times 2$ (viaje ida y vuelta) y la parte específica a la obtención de cuantos viajes adicionales son necesarios en función del tipo de mudanza.

- b) El diagrama de clases resultante de la aplicación del patrón método plantilla a las operaciones *getAccumulatedKM()* se encuentra a continuación.



- c) El diagrama de secuencia de la operación *getAccumulatedKM():Real* de la clase *Company* se encuentra a continuación:





Ejercicio 5 (25%)

Ahora supongamos que quieren un sistema de notificaciones para avisar a los operarios que tienen que participar en una mudanza y así notificarles todas las especificaciones: fecha, dirección de origen, de destino y todos los objetos que hay que mover. Cada vez que haya una modificación de fechas sobre una mudanza, el operario que esté asignado y el cliente recibirán un correo con el

texto siguiente: “La mudanza del día “ + startDate + “ de ” + “originAdress” + a + “destinationAdress” + contiene todos los siguientes paquetes para transportar + “list<Item>”. Podéis suponer la existencia de la operación sendMail(email:String, text:String) que puede ser invocada desde donde queráis y que envía el correo con el texto indicado en el parámetro a la dirección de correo indicada en el parámetro. Se pide:

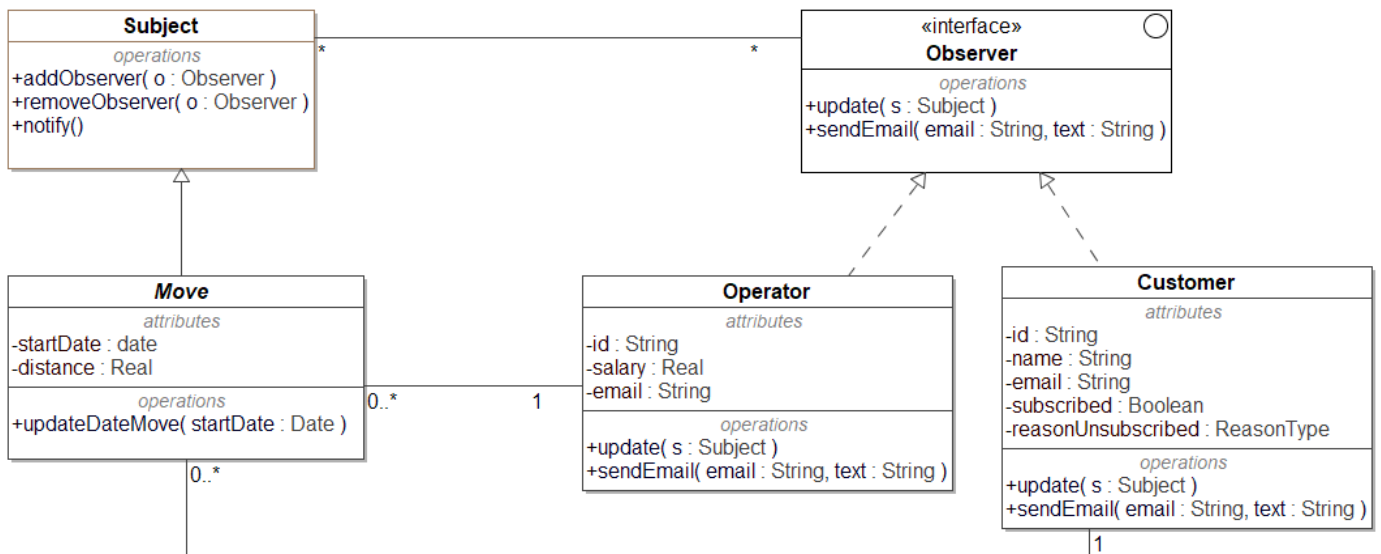
- a) ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- b) Haz el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar sólo la parte de la aplicación del patrón)
- c) Haz el diagrama de secuencia o el pseudocódigo de la operación *updateDateMove(startDate: Date)* de la clase *Move* que actualiza la fecha de inicio de la mudanza y notifica al operario y al cliente correspondiente de esta actualización. Haz también el diagrama de secuencia o pseudocódigo de todas las operaciones que sean invocadas desde esta operación.

Nota:

- Queremos evitar el acoplamiento entre los clientes y los operadores con sus mudanzas.

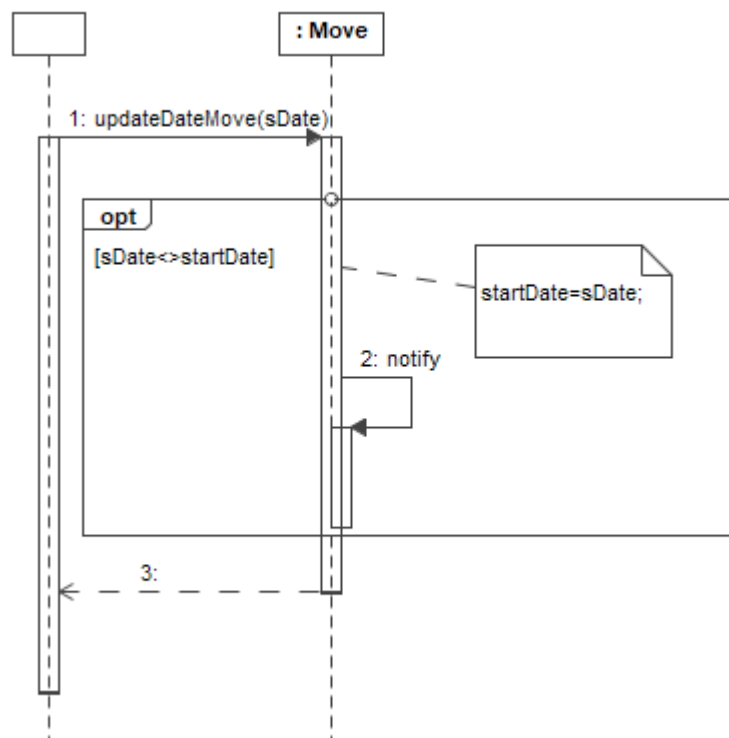
Solución:

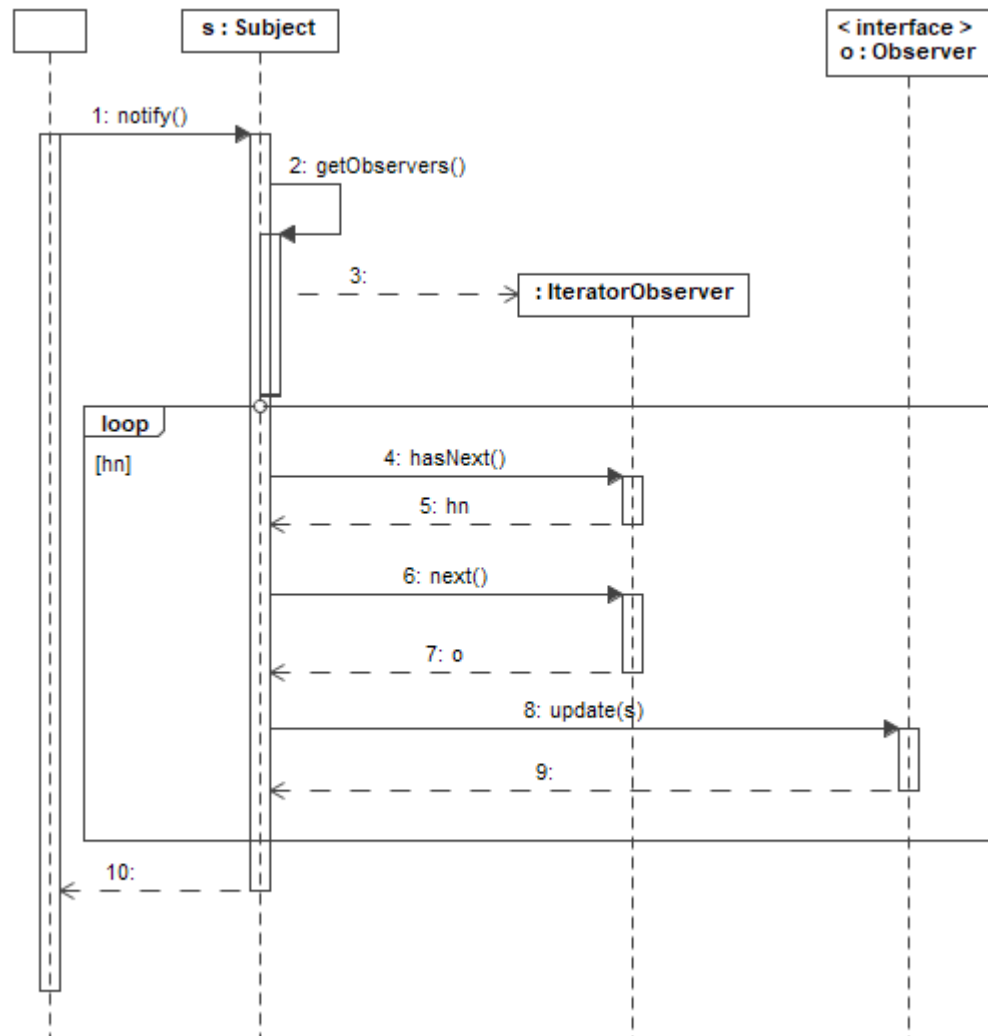
- a) El patrón principal que usaremos para tratar esta situación es el patrón Observador. Cuando llegue una modificación de la mudanza la operación notificar del patrón se encargará de actualizar todos los observadores (*Operator* y *Customer*) que estén asociados con el contenido. La operación de actualizar será la encargada de enviarles el correo.
- b) A continuación disponéis de la parte del diagrama de clases donde hemos aplicado el patrón Observador.

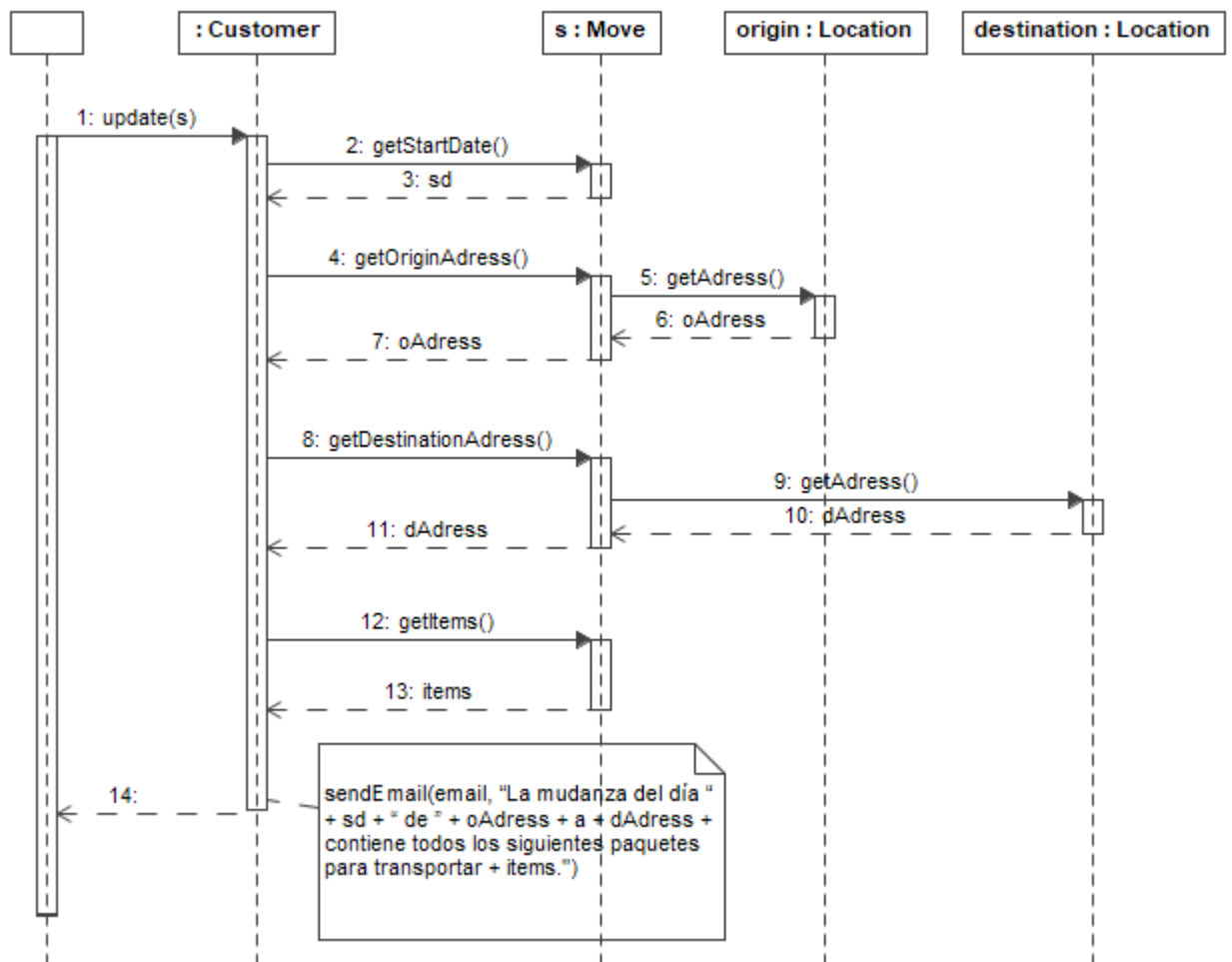


Las asociaciones entre las clases que ya existían se mantienen, se añaden las clases nuevas Subject y Observer para el patrón Observador y para la comunicación de la notificación y update.

c) A continuación se muestra el diagrama de secuencia de la operación *updateDateMove(startDate: Date)*.







La operación `update(s)` de `Operator` es la misma que la de `Customer`, sería replicar el diagrama anterior cambiando la clase `Customer` por `Operator`.